

Procesos Markovianos de Decisión

Daniela Pucci de Farias

Universidad Torcuato Di Tella

June 2025

Trabajo Práctico

- ▶ Finalizar formación de grupos
- ▶ 30 de junio: Entregar propuesta de proyecto y modelo inicial de MDP (máximo 2 páginas)

Clase 3

- ▶ MDPs con horizonte infinito
 - ▶ Ecuación de Bellman
 - ▶ Iteración de valor
 - ▶ Iteración de políticas
 - ▶ Función de utilidad
- ▶ Aplicaciones
 - ▶ Política de pesca
 - ▶ Optimización de portafolio

MDPs, Función Valor, Políticas

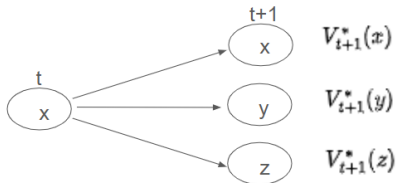
- ▶ **Proceso markoviano de decisión (MDP):**
 - ▶ tupla (S, P, r, A)
 - ▶ matriz de transición y recompensa dependen de la acción
 $a \in A: P_a, r_a$
- ▶ $u_t : S \mapsto A$ es una **política** indicando, para cada periodo t y estado x , una acción $a = u_t(x)$.
- ▶ **Función valor** V^* indica la recompensa esperada empezando en cada estado:

$$V_t^*(x) = \max_u \left[\sum_{\tau=t}^T \alpha^{\tau-t} r_u(x) | x_t = x \right]$$

- ▶ $\alpha \in [0, 1]$ es el factor de descuento

Programación Dinámica Para MDPs

- Calculamos la función valor V^* recursivamente:



Programación Dinámica con Horizonte T

1. Hacemos $t = T$. Calculamos $V_T^*(x) = \max_a r_a(x)$.
2. Hacemos $t = t - 1$. Calculamos
$$V_t^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_{t+1}^*(y) \right\}.$$
3. Si $t = 0$, fin. Si no, volvemos al paso 2.

- Derivamos la política óptima a partir de la función valor V^* :

$$u_t^*(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_{t+1}^*(y) \right\}$$

Ejemplo: Pesca de Salmones

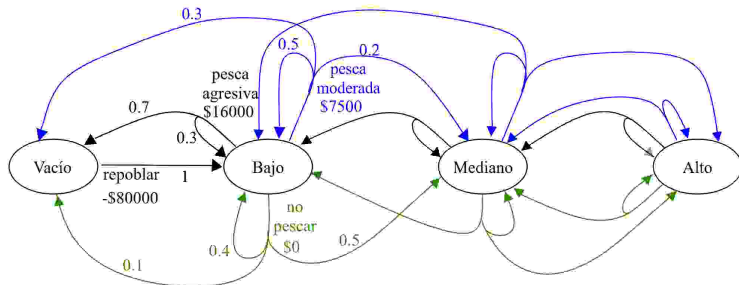
Cada año, necesitamos decidir cuánto salmón pescar en una determinada área. Cada salmón genera una ganancia fija. Pero si pescamos demasiados salmones en un año, eso impactará en la cantidad de salmones disponibles en el año siguiente. Debemos decidir cuánto pescar a cada año para maximizar la ganancia.

Cómo modelamos ese problema?

Vamos a considerar una versión simplificada para visualizar el proceso:

- ▶ a cada año, decidimos solo si pescar o no una cierta proporción de salmones;
- ▶ consideramos 4 estados para la población de salmones: vacío, bajo, mediano, alto.

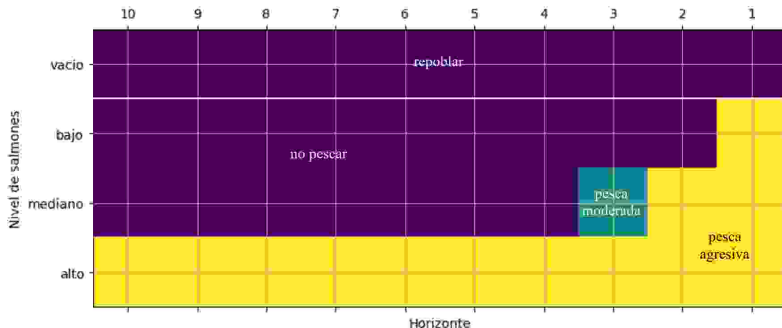
Pesca de Salmones: Modelo



Qué decisiones tomar? Vamos a resolver el problema con Python.

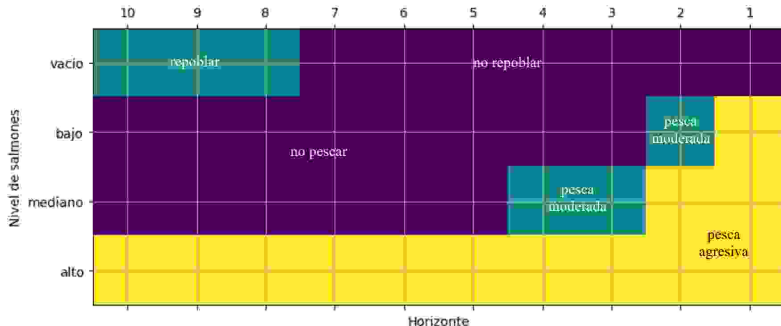
Pesca de Salmones: Resultados

- ▶ Aquí representamos la política óptima
 - ▶ Acción óptima depende del estado y del tiempo
- ▶ Cómo la elección del horizonte T impacta nuestras decisiones?



Pesca de Salmones: Opción de No Repoblar

- Si en el estado vacío tenemos la opción de no repoblar, cómo eso cambia el análisis? Cuáles son las condiciones para repoblar o no?



Pesca de Salmones: Horizonte

- ▶ Para obtener la política óptima, especificamos el horizonte T .
 - ▶ La acción óptima depende de la distancia al horizonte.
 - ▶ Qué horizonte debemos elegir?
 - ▶ En algunas situaciones, la elección del horizonte es clara:
 - ▶ Subastas
 - ▶ Campaña electoral
 - ▶ Logística y operaciones con fecha de entrega
 - ▶ Precificación de pasajes aéreos
- ▶ Qué hacer si no hay una elección clara?
- ▶ Podemos utilizar el horizonte infinito.

MDPs con Horizonte Infinito

- ▶ Vamos a considerar la ganancia esperada con $T = \infty$ y factor de descuento α :

$$\max_u \mathbb{E} \left[\sum_{t=0}^{\infty} \alpha^t r_u(x_t) \right]$$

- ▶ factor de descuento α introduce preferencia entre ganancias próximas y ganancias futuras
- ▶ también puede ser necesario para garantizar convergencia de la suma de ganancias
- ▶ en muchas situaciones, se puede trabajar con $\alpha = 1$ y considerar la ganancia promedio

$$\max_u \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=0}^{T-1} r_u(x_t) \right]$$

- ▶ Cómo encontrar la política óptima?

MDPs con Horizonte Infinito

- ▶ Consideremos nuevamente la recursión utilizada con horizonte finito, con descuento.
- ▶ Qué sucede cuando $T \mapsto \infty$? $\lim_{T \mapsto \infty} V_t^* = V^*, \forall t$.
- ▶ Ecuación de Bellman:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- ▶ Si logramos resolver la ecuación de Bellman, podemos derivar nuevamente la política óptima a partir de la función valor:

$$u^*(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- ▶ Notamos que u^* no depende de t , la política óptima es estacionaria.

Ecuación de Bellman - Iteración de Valor

- ▶ Hay varios algoritmos para resolver la ecuación de Bellman, nos enfocaremos en dos métodos que forman la base para los algoritmos de aprendizaje por refuerzo, iteración de valor e iteración de políticas.
- ▶ Iteración de valor es básicamente el mismo algoritmo que el algoritmo de programación dinámica para horizonte finito:

Iteración de Valor

1. Hacemos $t = 0$. Inicializamos $V_0(x)$ con un valor arbitrario, por ejemplo $V_0(x) = 0 \forall x$.
2. Hacemos $t = t + 1$. Calculamos
$$V_{t+1}(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_t(y) \right\}.$$
3. Utilizando un criterio de convergencia para V_t , decidimos si parar o volver al paso 2.
 - ▶ Por ejemplo, $\max_x |V_t(x) - V_{t-1}(x)| < \epsilon$ para algún $\epsilon > 0$

Ecuación de Bellman - Iteración de Política

Iteración de Política

1. Hacemos $k = 0$. Empezamos con una política arbitraria u_0 .
2. Evaluamos la función valor V_{u_k} asociada con la política u_k . Para eso, debemos resolver:

$$V_{u_k} = r_{u_k} + \alpha P_{u_k} V_{u_k}$$

Calculamos una nueva política u_{k+1} , que será la **política codiciosa** con respecto a V_{u_k} :

$$u_{k+1}(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_{u_k}(y) \right\}$$

3. Si $u_k = u_{k+1}$, paramos. Si no, hacemos $t = t + 1$ y volvemos al paso 2.

Ecuación de Bellman - Iteración de Política - Intuición:

- ▶ Tenemos

$$u_{k+1}(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_{u_k}(y) \right\}$$

entonces:

$$r_{u_{k+1}} + \alpha P_{u_{k+1}} V_{u_k} \geq r_{u_k} + \alpha P_{u_k} V_{u_k} = V_{u_k}$$

$$\underbrace{r_{u_{k+1}} + \alpha P_{u_{k+1}} V_{u_k}}_{\text{valor de seguir } (u_{k+1}, u_k, u_k, \dots)} \geq \underbrace{r_{u_k} + \alpha P_{u_k} V_{u_k}}_{\text{valor de seguir } (u_k, u_k, u_k, \dots)} = V_{u_k}$$

- ▶ La desigualdad es estricta por lo menos para un estado x , si u_k no es una política óptima $\Rightarrow$ La política (no estacionaria) $(u_{k+1}, u_k, u_k, \dots)$ es mejor que $(u_k, u_k, u_k, \dots)$.
- ▶ Siguiendo el mismo razonamiento, podemos demostrar que $(u_{k+1}, u_{k+1}, u_k, u_k, \dots)$ es mejor que $(u_{k+1}, u_k, u_k, u_k, \dots)$.
- ▶ Podemos seguir reemplazando u_k con u_{k+1} sucesivamente, obteniendo una secuencia de políticas que van mejorando.
- ▶ Concluimos que u_{k+1} es mejor que u_k .

Políticas Rollout

- ▶ Consideremos una única iteración de política:
 1. Evaluamos la función valor V_u asociada con una política u
 - ▶ u puede ser una heurística que ya se utiliza
 - ▶ u y/o V_u pueden ser el resultado de aprendizaje por refuerzo
 2. Calculamos una nueva política u_{rollout} , que será la **política codiciosa** con respecto a V_u :

$$u_{\text{rollout}}(x) = \arg \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V_{u_k}(y) \right\}$$

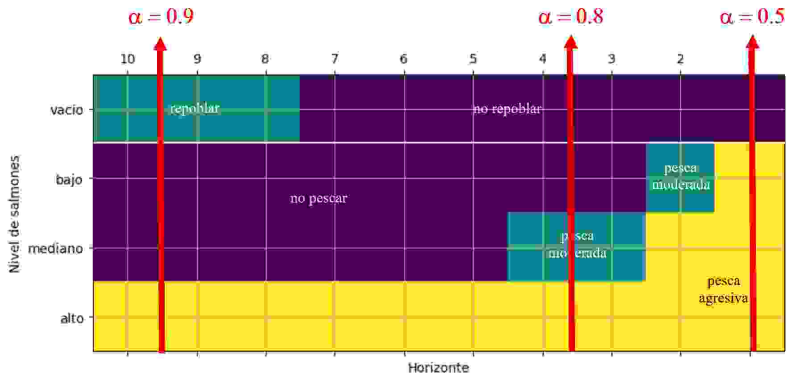
- ▶ Sabemos que u_{rollout} es mejor que u .
 - ▶ Opción práctica para mejorar heurísticas.
 - ▶ Utilizado en aplicaciones exitosas de aprendizaje por refuerzo, como TD-Gammon y AlphaGo

Iteración de Valor vs Iteración de Política

- ▶ Como hay un número finito de políticas, iteración de política siempre termina.
- ▶ El número de políticas estacionarias es $|A|^{|S|}$. En el peor caso, la iteración de política necesita un número de iteraciones que es exponencial en $|S|$, pero en la práctica, muchas veces termina en pocos pasos.
- ▶ Un paso de iteración de política es más costoso que un paso de iteración de valor, porque debemos evaluar la política y seleccionar la nueva acción.
- ▶ Sin embargo, la iteración de política suele terminar en menos pasos, y la iteración de valor puede ser lenta, especialmente cuando el factor de descuento es alto.
- ▶ Iteración de valor e iteración de política forman la base para muchos algoritmos de aprendizaje por refuerzo.

Pesca de Salmones: Horizonte Infinito

Resolución con horizonte infinito, distintos factores de descuento.
Implementamos iteración de valor en Python...



Ejemplo: Optimización de Portafolio

- ▶ Consideramos una situación donde hay dos posibilidades de inversión:
 - ▶ una caja de ahorros con tasa de interés r_f ;
 - ▶ un activo con precio s_t que evoluciona en el tiempo según

$$s_{t+\Delta} = s_t e^{(r-\sigma^2/2)\Delta + \sigma\sqrt{\Delta}\epsilon_t}$$

donde ϵ_t , $t = 0, 1, \dots$ son variables aleatorias independientes y siguen la distribución normal estándar ($\epsilon_t \sim N(0, 1)$).

- ▶ Ese es el tradicional **modelo lognormal** para precios de activos.
 - ▶ r es el rendimiento esperado del activo y σ es su volatilidad.
- ▶ Cómo debemos invertir para maximizar el valor del portafolio después de un periodo T ?

Modelo Lognormal Para Precios - Tiempo Discreto

- ▶ El modelo lognormal para precios es válido para cualquier t y Δ : $s_{t+\Delta} = s_t \cdot e^{(r-\sigma^2/2)\Delta + \sigma\sqrt{\Delta}\epsilon_t}$ donde ϵ_t , $t = 0, 1, \dots$ son variables aleatorias independientes y siguen la distribución normal estándar ($\epsilon_t \sim N(0, 1)$).
- ▶ Vamos a considerar el modelo a tiempo discreto, con $t = 0, \Delta, 2\Delta, \dots$ para un valor fijo de Δ . Consideremos que tomamos decisiones solamente en esos momentos discretos.
- ▶ Noten que:

$$\begin{aligned} s_{t+\Delta} &= s_t e^{(r-\sigma^2/2)\Delta + \sigma\sqrt{\Delta}\epsilon_t} \\ &= s_t e^{(r\Delta - (\sigma\sqrt{\Delta})^2/2 + \sigma\sqrt{\Delta}\epsilon_t)} \\ &= s_t e^{\tilde{r} - \tilde{\sigma}^2/2 + \tilde{\sigma}\epsilon_t} \end{aligned}$$

donde $\tilde{r} = r\Delta$ y $\tilde{\sigma} = \sigma\sqrt{\Delta}$.

- ▶ Entonces podemos considerar el modelo a tiempo discreto:

$$\tilde{s}_{n+1} = \tilde{s}_n e^{\tilde{r} - \tilde{\sigma}^2/2 + \tilde{\sigma}\epsilon_n}$$

donde $\epsilon_n \sim N(0, 1)$ y $\tilde{s}_n = s_{n\Delta}$.

Modelo Lognormal Para Precios - Tiempo Discreto

- Modelo a tiempo discreto:

$$s_{n+1} = s_n e^{r - \sigma^2/2 + \sigma \epsilon_n}$$

donde $\epsilon_n \sim N(0, 1)$ y $\tilde{s}_n = s_n \Delta$. (Sacamos los tildes para simplificar la notación.)

- Ejemplo: Si un activo tiene rendimiento esperado anual de 10% y volatilidad de 40%, y consideramos rebalancear el portafolio una vez por mes, cómo modelamos su precio?
 - Cada periodo corresponde a $\Delta = 1/12$ (un mes).
 - El rendimiento mensual tiene valor esperado y volatilidad de

$$r = 0.1/12 \approx 0.0083, \quad \sigma = 0.4\sqrt{1/12} \approx 0.1155$$

- La evolución del precio del activo mes a mes es dada por

$$s_{t+1} = s_t e^{0.0083 - 0.1155^2/2 + 0.1155 \epsilon_t} = s_t e^{0.0016 + 0.1155 \epsilon_t}.$$

Optimización de Portafolio: Modelo MDP (S, A, P, r)

- ▶ Estado: valor W_t del portafolio
- ▶ Acción: fracción β_t del portafolio invertida en la acción
- ▶ Transición: $W_{t+1} = W_t \left(\beta_t e^{r - \sigma^2/2 + \sigma \epsilon_t} + (1 - \beta_t) e^{r_f} \right)$
- ▶ Recompensa/Criterio de optimalidad?
 - ▶ Empezamos maximizando el valor del portafolio al final de T periodos:

$$\max_u E[W_T]$$

- ▶ $r_T(w) = w$ $r_t(w) = 0, \forall t < T$.
- ▶ Maximizar el valor final esperado del portafolio es solo una opción, hay muchas formulaciones posibles.

Optimización de Portafolio: Función Valor y Recursión

- ▶ La función valor es $V_t^*(w) = E[W_T | W_t = w]$
 - ▶ valor esperado del portafolio en el periodo T dado el valor actual w .
- ▶ La ecuación de Bellman es:

$$\begin{aligned} V_T^*(w) &= w \\ V_t^*(w) &= \max_{\beta \in [0,1]} E[V_{t+1}^*(W_{t+1}) | W_t = w] \\ &= \max_{\beta \in [0,1]} E\left[V_{t+1}^*\left(w \left[\beta e^{r - \sigma^2/2 + \sigma\epsilon_t} + (1 - \beta)e^{r_f}\right]\right)\right] \end{aligned}$$

donde utilizamos la transición

$$W_{t+1} = W_t \left(\beta_t e^{r - \sigma^2/2 + \sigma\epsilon_t} + (1 - \beta_t)e^{r_f} \right).$$

- ▶ En este ejemplo, el espacio de estados es continuo y la transición está representada por una función, en vez de la matriz de transición P .
 - ▶ Debemos evaluar $E[V_{t+1}^*(w_{t+1})]$, que en este caso no se expresa como $\sum_y P_{\beta}(w, w') V_{t+1}^*(w')$.

Optimización de Portafolio: Solución

Vamos a utilizar recursión, ya que el horizonte es finito.

- ▶ En el periodo final, recibimos $V_T^*(w) = w$.
- ▶ En $t = T - 1$, debemos optimizar:

$$\begin{aligned} V_{T-1}^*(w) &= \max_{\beta \in [0,1]} \mathbb{E} \left[V_T^* \left(w \left[\beta e^{r - \sigma^2/2 + \sigma \epsilon_{T-1}} + (1 - \beta) e^{r_f} \right] \right) \right] \\ &= \max_{\beta \in [0,1]} \mathbb{E} \left[w \left(\beta e^{r - \sigma^2/2 + \sigma \epsilon_{T-1}} + (1 - \beta) e^{r_f} \right) \right] \\ &= w \max_{\beta \in [0,1]} \mathbb{E} \left[\left(\beta e^{r - \sigma^2/2 + \sigma \epsilon_{T-1}} + (1 - \beta) e^{r_f} \right) \right] \\ &= w \max_{\beta \in [0,1]} \left\{ \beta e^{r - \sigma^2/2} \mathbb{E} [e^{\sigma \epsilon_{T-1}}] + (1 - \beta) e^{r_f} \right\} \\ &= w \max_{\beta \in [0,1]} \left\{ \beta e^{r - \sigma^2/2} \underbrace{\mathbb{E} [e^{\sigma \epsilon_{T-1}}]}_{e^{\sigma^2/2}} + (1 - \beta) e^{r_f} \right\} \\ &= w \max_{\beta \in [0,1]} \left\{ \beta e^r + (1 - \beta) e^{r_f} \right\} \end{aligned}$$

- ▶ Tenemos:
 - ▶ $\beta_{T-1}^*(w) = 1$ si $r > r_f$ y 0 si $r < r_f$, independiente de w ;
 - ▶ $V_{T-1}^*(w) = w \max(e^r, e^{r_f}) = wC$
 - ▶ V_{T-1}^* es lineal en w .

Optimización de Portafolio: Solución

Vamos a utilizar recursión, ya que el horizonte es finito.

- ▶ En el periodo final, recibimos $V_T^*(w) = w$.
- ▶ En $t = T - 2$, debemos optimizar:

$$\begin{aligned} V_{T-2}^*(w) &= \max_{\beta \in [0,1]} \mathbb{E} \left[V_{T-1}^* \left(w \left[\beta e^{r - \sigma^2/2 + \sigma \epsilon_{T-2}} + (1 - \beta) e^{r_f} \right] \right) \right] \\ &= \max_{\beta \in [0,1]} \mathbb{E} \left[C w \left(\beta e^{r - \sigma^2/2 + \sigma \epsilon_{T-2}} + (1 - \beta) e^{r_f} \right) \right] \\ &= C w \max_{\beta \in [0,1]} \mathbb{E} \left[\left(\beta e^{r - \sigma^2/2 + \sigma \epsilon_{T-2}} + (1 - \beta) e^{r_f} \right) \right] \\ &= C w \max_{\beta \in [0,1]} \left\{ \beta e^{r - \sigma^2/2} \mathbb{E} [e^{\sigma \epsilon_{T-2}}] + (1 - \beta) e^{r_f} \right\} \\ &= C w \max_{\beta \in [0,1]} \{ \beta e^r + (1 - \beta) e^{r_f} \} \end{aligned}$$

▶ Tenemos:

- ▶ $\beta_{T-2}^*(w) = 1$ if $r > r_f$ y 0 si $r < r_f$, independiente de w ;;
- ▶ $V_{T-2}^*(w) = C w \max(e^r, e^{r_f}) = C^2 w$
 - ▶ V_{T-2}^* es lineal en w .
 - ▶ Podemos seguir el mismo argumento con $t = T - 3, \dots$.

Optimización de Portafolio: Solución

- Del desarrollo anterior, concluimos que la función valor tiene la forma:

$$V_t^*(w) = w \max(e^r, e^{r_f})^{T-t}.$$

- Entonces, la política óptima no depende del tiempo ni del valor del portafolio:

$$\beta_t^*(w) = \beta^* = \begin{cases} 1 & \text{si } r > r_f \\ 0 & \text{si } r < r_f \\ \text{cualquier valor} & \text{si } r = r_f \end{cases}$$

Optimización de Portafolio: Observaciones

- ▶ En ese ejemplo, la solución es simple e intuitiva, reduciéndose a maximizar el rendimiento esperado en cada período.
 - ▶ Eso se debe a que el rendimiento del activo es independiente y distribuido igualmente a cada periodo.
- ▶ La metodología puede ser aplicada en condiciones más complejas, por ejemplo:
 - ▶ la tasa de interés, el rendimiento de la acción y su volatilidad pueden ser fluctuantes;
 - ▶ pueden haber costos para realizar transacciones.
- ▶ En este caso tanto la variable de estado como la acción son continuas.
 - ▶ Si una solución analítica no existe, necesitamos realizar aproximaciones, por ejemplo discretizando el espacio de estados y/o el espacio de acciones.

Repensando La Función Recompensa

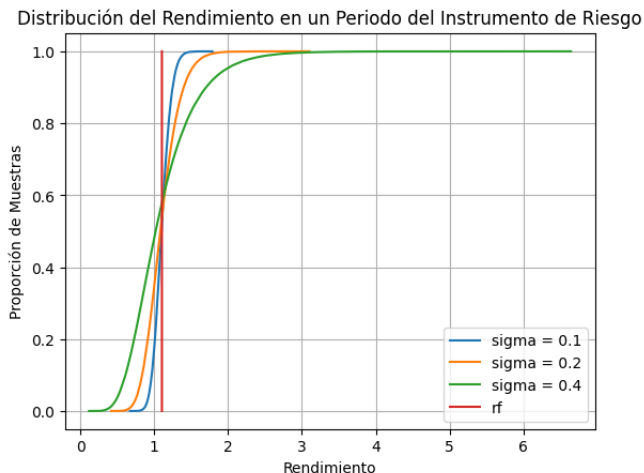
- ▶ Maximizando el valor esperado final $E[W_T]$, la optimización en cada período t se reduce a

$$\max_{\beta \in [0,1]} \{\beta e^r + (1 - \beta)e^{r_f}\}$$

- ▶ Comparación simple entre el rendimiento esperado r del activo y la tasa de interés r_f .
- ▶ Independiente de la volatilidad σ del instrumento de riesgo.
- ▶ Pero: $R_{\text{riesgo}} = e^{r - \sigma^2/2 + \sigma\epsilon}$, donde $\epsilon \sim N(0,1)$, y $R_{\text{ahorro}} = e^{r_f}$
- ▶ Vamos a comparar R_{riesgo} y R_{ahorro} con distintos valores de $\sigma \dots$

Repensando La Función Recompensa

- ▶ $R_{\text{riesgo}} = e^{r - \sigma^2/2 + \sigma\epsilon}$, , donde $\epsilon \sim N(0, 1)$, y $R_{\text{ahorro}} = e^{r_f}$
- ▶ Generamos la distribución de R_{riesgo} con Python...



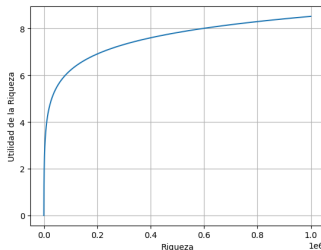
- ▶ Somos indiferentes a esas cuatro opciones?

Aversión al Riesgo

- ▶ Consideremos una situación donde podés elegir recibir un monto fijo o un monto establecido por el resultado de tirar una moneda.
- ▶ Si el monto fijo es \$500 y el monto dado por la moneda es \$0 o \$1000, qué elegís?
- ▶ Si el monto fijo es US\$500,000 y el monto dado por la moneda es US\$0 o US\$1,000,000, qué elegís?

Función de Utilidad

- ▶ Podemos capturar nuestras preferencias respecto al riesgo eligiendo la función de recompensa con el concepto de **función de utilidad**.
- ▶ La función de utilidad captura el valor de distintas opciones. Por ejemplo, en el caso del juego con la moneda, podemos elegir $\nu(w) = \log(0.001w + 1)$.



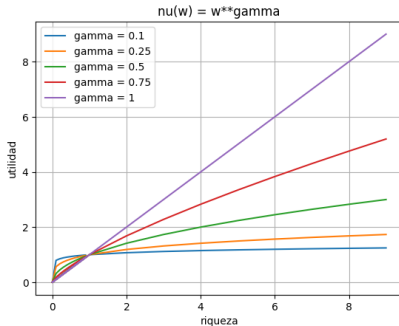
- ▶ Por qué cóncava?
- ▶ Entonces:

$$u(0.5) \approx 0.5(u(0) + u(1)) \approx 0.0005$$

$$u(500000) = 6.2166 > 0.5(u(0) + u(1000000)) = 3.4544$$

Función de Utilidad

- ▶ Vamos a aplicar la función de utilidad $u(w) = w^\gamma / \gamma$, donde $0 < \gamma \leq 1$.
- ▶ Funciones con **aversión relativa constante al riesgo**
 - ▶ $u(aw) = a^\gamma u(w)$
 - ▶ preferencia relativa $u(aw)/u(w) = a^\gamma / \gamma$ no depende de w .
- ▶ Casos límite:
 - ▶ Con $\gamma = 1$, $u(w) = w$, utilidad neutral al riesgo
 - ▶ $\lim_{\gamma \rightarrow 0} u(w) = \log(w)$



Optimización de Portafolio con Función de Utilidad

- ▶ Nuestro criterio se vuelve: $\max_u E[\nu(W_T)] = \max_u E[W_T^\gamma]$.
- ▶ Podemos seguir los mismos pasos que utilizamos en la maximización del valor esperado $E[W_T]$ para determinar que:

$$V_t^*(w) = w^\gamma \left\{ \max_{\beta \in [0,1]} E_{\epsilon \sim N(0,1)} \left[\left(\beta e^{r - \sigma^2/2 + \sigma\epsilon} + (1 - \beta)e^{r_f} \right)^\gamma \right] \right\}^{T-t}$$

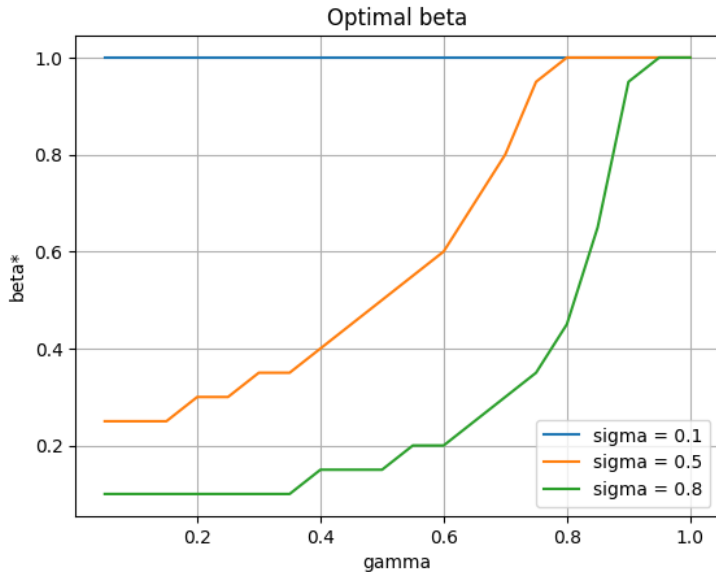
- ▶ Entonces, la política óptima es constante en el tiempo e independiente de la riqueza:

$$\beta_{\gamma,t}^*(w) = \beta_\gamma^* = \arg \max_{\beta \in [0,1]} E_{\epsilon \sim N(0,1)} \left[\left(\beta e^{r - \sigma^2/2 + \sigma\epsilon} + (1 - \beta)e^{r_f} \right)^\gamma \right]$$

- ▶ Calculamos β_γ^* con Python...

Optimización de Portafolio con Función de Utilidad

► $r_f = 0.04$, $r = 0.1$



Optimización de Portafolio con Consumo

- ▶ Ahora asumimos que podemos gastar una parte de la riqueza a cada periodo.
- ▶ Modelo MDP?
 - ▶ Estado = valor W_t del portafolio
 - ▶ Acción: $0 \leq \beta_t \leq 1, 0 \leq \delta_t \leq 1$
 - ▶ En cada periodo t , consumimos $\delta_t W_t$ e invertimos una fracción β_t del restante en el activo y $1 - \beta_t$ en la caja de ahorros.
 - ▶ Transición: $W_{t+1} = (1 - \delta_t)W_t \left[\beta_t e^{r - \sigma^2/2 + \sigma\epsilon_t} + (1 - \beta_t)e^{r_f} \right]$
 - ▶ Recompensa: $\nu(\delta_t W_t)$
 - ▶ Criterio de optimalidad: $\max_u \left[\sum_{t=0}^T \alpha^t \nu(\delta_t W_t) \right]$.
- ▶ Ecuación del Bellman:

$$V^*(w) = \max_{\beta, \delta} \left\{ \nu(\delta w) + \alpha E_{\epsilon \sim N(0,1)} \left[V^* \left((1 - \delta)w \left[\beta e^{r - \sigma^2/2 + \sigma\epsilon} + (1 - \beta)e^{r_f} \right] \right) \right] \right\}$$